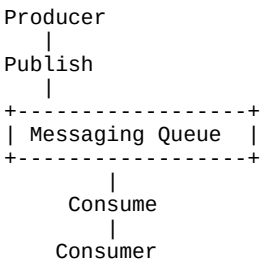
Distributed Messaging Queue - SDE1 HLD Detailed Study Guide

What is a Messaging Queue

A messaging queue is middleware that allows independent applications or microservices to communicate by exchanging messages asynchronously. Instead of calling another service directly and waiting for a response, a producer places a message into a queue. The queue stores the message until a consumer is available to process it. This decouples services, improves reliability and allows producers and consumers to scale independently. The queue acts as a temporary buffer during traffic spikes.

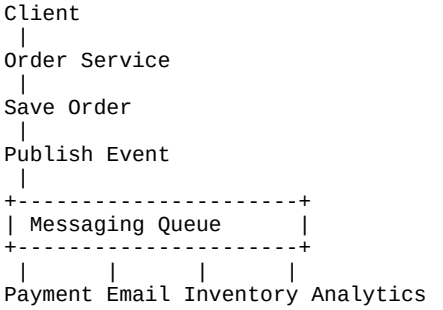
Architecture



Why is it Needed?

In distributed systems, directly calling every downstream service increases latency and tightly couples services. A messaging queue allows the producer to return a response immediately while background work continues asynchronously. It also absorbs sudden traffic spikes, prevents cascading failures, supports retries and improves fault tolerance. Modern systems such as e-commerce, banking and ride-sharing platforms use queues for emails, notifications, payments, analytics and order processing.

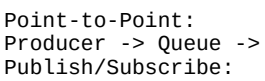
Architecture

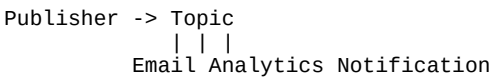


Point-to-Point vs Publish/Subscribe

Point-to-Point (P2P) delivers each message to exactly one consumer. It is best suited for task processing where duplicate execution is undesirable, such as payment processing.
Publish/Subscribe broadcasts an event to every interested subscriber. Each subscriber receives its own copy of the event and performs an independent action. Pub/Sub is widely used in event-driven architectures.

Architecture

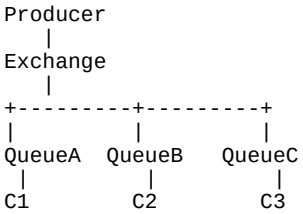




RabbitMQ

RabbitMQ is a general-purpose message broker implementing the AMQP protocol. Producers never send messages directly to queues. Instead, messages first arrive at an Exchange, which decides where they should go based on routing rules. Consumers acknowledge successful processing; only then are messages removed. RabbitMQ excels at task queues, RPC-style messaging and flexible routing.

Architecture



RabbitMQ Components

Producer publishes messages. Exchange receives messages and applies routing logic. Bindings connect exchanges to queues. Routing Keys decide the destination for Direct and Topic exchanges. Queues store messages until consumers process them. Consumers read messages and send acknowledgements (ACK). The Broker hosts exchanges and queues, while Virtual Hosts isolate multiple applications on the same broker.

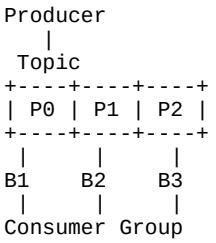
Exchange Types

Direct Exchange routes using an exact routing key. Fanout Exchange ignores routing keys and broadcasts to every bound queue. Topic Exchange supports wildcard routing such as order.* or *.created, making it ideal for event-driven systems. Headers Exchange routes using message headers instead of routing keys and is used less frequently.

Kafka

Kafka is a distributed event streaming platform optimized for extremely high throughput. Producers write events to Topics, which are divided into Partitions spread across multiple Brokers. Consumers read messages using Offsets, allowing them to replay historical data. Unlike RabbitMQ, Kafka retains messages for a configurable time even after they are consumed.

Architecture



Kafka Components

Producer writes events. Broker stores partitions. Topics logically group events. Partitions enable parallel processing and horizontal scaling. Consumer Groups divide work among multiple consumers. Offsets identify the position of each consumed message. Replication copies partitions across brokers for fault tolerance, while the Controller/KRaft manages metadata and leader election.

Failure Scenarios

If the queue becomes full, the broker may reject producers, block producers, expire old messages or redirect them to a Dead Letter Queue depending on configuration. If a broker fails, replicated brokers elect a new leader to continue processing. If a consumer crashes before acknowledging a message, the broker redelivers it. If a consumer repeatedly fails to process a message, it is retried and finally moved to a Dead Letter Queue so that bad messages do not block the system.

Interview Questions with Answers

1. Why use a messaging queue? It decouples services, improves responsiveness, enables asynchronous processing and provides reliability through retries.

2. Queue vs Pub/Sub? Queue sends each message to one consumer; Pub/Sub sends a copy to every subscriber.

3. Why ACK? ACK guarantees a message is deleted only after successful processing.

4. Why partitions? They enable parallelism and horizontal scalability.

5. What happens if a consumer crashes? The message remains unacknowledged and is redelivered.

6. What is a DLQ? A queue that stores messages which fail repeatedly after retries.

7. Kafka vs RabbitMQ? Kafka is optimized for event streaming and message retention; RabbitMQ is optimized for task queues and flexible routing.